TRAITEMENT DE DONNÉES · FICHE DE RÉVISION · TD 2

Représentation des images

RGB vs LAB, dictionnaire K-Means, histogramme, mAP, √x, IDF

À RETENIR EN PRIORITÉ

- Une image = tableau **(hauteur, largeur, 3)** en `uint8` (0–255). `I.reshape((-1, 3))` → liste de pixels **(H×W, 3)**.
- RGB 24 bits = 256^3 **= 16 777 216** couleurs : trop pour un histogramme → **dictionnaire de P = 128 couleurs** appris par **K-Means**.
- RGB n'est **pas perceptuellement uniforme** ; en **LAB**, la distance euclidienne ≈ écart de couleur perçu → clustering et distances en LAB.
- **Histogramme couleur** = proportion de pixels associés à chaque couleur du dictionnaire (vecteur de taille **P**, somme 1). C'est le **sac-de-mots** des images.
- Évaluation sur Holidays (1491 images, 500 groupes, 500 requêtes) par le **mAP** : brut **40,43 %** → √x **54,91 %** → √x × idf **55,29 %**.
- **√x** sature les couleurs dominantes (comme BM25) ; **idf(k) = ln(1 / df(k))** avec df(k) = **proportion moyenne** de la couleur k.

1. Une image en NumPy

`images[42].shape` → `(192, 256, 3)` : **192 lignes (hauteur)**, **256 colonnes (largeur)**, **3 canaux R, G, B**, chacun en `uint8` entre 0 et 255.

Opération	Forme obtenue
`I`	(192, 256, 3)
`I.reshape((-1, 3))`	(49 152, 3) : un pixel par ligne
`I.size`	147 456 valeurs (= 49 152 × 3)
`rgb_samples` (échantillon)	(100 000, 3)

PIÈGE : LE UINT8 BOUCLE

En `uint8`, les calculs se font **modulo 256** : `5 - 35` donne **226**, pas −30. D'où les `astype(float)` / `astype(np.float64)` avant tout calcul de différence ou de distance.
Et l'ordre est **hauteur puis largeur** (lignes, colonnes), pas l'inverse.

2. Pourquoi l'espace LAB plutôt que RGB

Pour construire le dictionnaire il faut une **distance entre couleurs**. La distance euclidienne en RGB ne suit pas l'écart **perçu par l'œil** : une même distance peut être invisible dans une zone de l'espace et nette dans une autre.

CIE LAB est construit pour que la **distance euclidienne approxime la différence perçue** partout. On fait donc K-Means et `predict` en LAB.

- `rgb2lab` attend des valeurs dans **[0, 1]** et une forme **(h, w, 3)** → `rgb.reshape(-1, 1, 3) / 255.0`, puis `.reshape(-1, 3)`.
- `lab2rgb` sert à **afficher** (un écran n'affiche que du RGB) ; il renvoie [0, 1] → `* 255`, `clip(0, 255)`, `astype(np.uint8)`.

EXEMPLE DU TD : MÊME DISTANCE RGB

```
Paire A (cyans)  : (35,239,234) / (5,239,236)
  écarts (30, 0, -2)
  d = √(900 + 0 + 4)     = √904 ≈ 30,1
Paire B (bleus)  : (3,12,70) / (5,36,52)
  écarts (-2, -24, 18)
  d = √(4 + 576 + 324)   = √904 ≈ 30,1
```

Même d_{RGB} ≈ 30,1, mais A est **indiscernable** à l'œil et B **nettement différente** : RGB n'est pas perceptuellement uniforme.

3. Le dictionnaire de couleurs (K-Means)

1. **Échantillon** : 100 000 pixels tirés au hasard (sans remise) parmi plus de **70 millions** — K-Means sur tout serait trop coûteux.
2. **Conversion** en LAB → `lab_samples` (100 000, 3).
3. `KMeans(P=128, n_init=1, random_state=42).fit(lab_samples)` : les **128 centres** (`cluster_centers_`, points LAB, moyennes de pixels) = les **128 couleurs du dictionnaire**.
4. `palette_rgb` : centres reconvertis en RGB 0–255, forme (128, 3).
5. **Quantification** : `kms.predict` associe chaque pixel (en LAB) à la couleur la plus proche ; `palette_rgb[idx].reshape(H, W, 3)` redonne une image d'au plus 128 couleurs.

`n_init` = nombre de **lancements** de K-Means avec des initialisations différentes (on garde la plus faible inertie). 1 pour aller vite ; en pratique plus.

Texte (sac-de-mots)	Images (TD 2)
Vocabulaire	**Dictionnaire de P couleurs**
Document	Image
Occurrence d'un mot	Pixel associé à une couleur
Sac-de-mots	**Histogramme couleur**
BM25 (saturation)	$\sqrt{x}$
IDF (df = nb de documents)	IDF (df = **proportion moyenne**)

Visualisation de la palette (`plot_palette_hsv`) : **angle = teinte**, **rayon = saturation**, **taille = valeur** (HSV).

4. L'histogramme couleur

```
def compute_hist(kmeans, I):
    I_lab = rgb2lab(I.astype(np.float64) / 255.0)
    idx = kmeans.predict(I_lab.reshape((-1, 3)))
    x = np.zeros((kmeans.n_clusters,))
    idv, v = np.unique(idx, return_counts=True)
    x[idv] = v                # comptes à leur place
    return x / np.sum(x)      # proportions (somme 1)
```

TUTO : CALCULER UN HISTOGRAMME

1. Vecteur de **P zéros** (une case par couleur, même absente).
2. Compter les pixels de chaque couleur et les ranger à leur indice.
3. **Diviser par le nombre de pixels.**

Exemple : P = 5, `idx = [0, 2, 2, 3, 2, 0]`
comptes [2, 0, 3, 1, 0] → **[1/3, 0, 1/2, 1/6, 0]**

`X` = histogrammes de toutes les images : forme **(1491, 128)**. La **normalisation** rend l'histogramme indépendant de la taille : 2 500 px sur 100×100 et 10 000 px sur 200×200 donnent tous deux **0,25**.

5. Recherche par similarité et mAP

Holidays : 1491 photos, 500 groupes (scène ou objet). La **1re image** de chaque groupe est la requête ; but : retrouver en tête les autres images du groupe.

- Nom `112301.jpg` : groupe `int(f[1:4])` = **123** ; requête si le nom finit par `00.jpg` (ici **non**).
- `dist = euclidean_distances(X[queries], X)` : forme **(500, 1491)**, triée par `argsort`.
- Dans `score_map`, la **requête est exclue** des résultats (distance 0 à elle-même, sinon toujours 1re).
- **mAP** = moyenne des AP des 500 requêtes (× 100).

TUTO : AP DE `SCORE_AP_FROM_RANKS_1`

Pour chaque bon résultat (rang `r` à partir de 0, `ntp` bons avant lui) :
p_0 = ntp / r (1 si r = 0), p_1 = (ntp + 1) / (r + 1)
AP += (p_0 + p_1) / 2 × 1 / nres (**trapèzes**)

```
nres = 2, rangs [0, 2]
 r=0 : (1   + 1  )/2 × 1/2 = 0,500
 r=2 : (1/2 + 2/3)/2 × 1/2 ≈ 0,292
 AP ≈ 0,79    (AP classique : 0,83)
nres = 1, rang [1]
 r=1 : (0 + 1/2)/2 × 1     = 0,25
```

6. Améliorer la représentation : √x puis IDF

Racine carrée (saturation, comme BM25)

`X_sqrt = np.sqrt(X)`. √ croît vite près de 0 puis s'aplatit : une couleur qui domine (ciel) ne **écrase** plus les autres. 0,64 et 0,01 → 0,8 et 0,1 : le rapport passe de **64 à 8**.

Exemple : q = [0,8 ; 0,1 ; 0,1 ; 0], A = [0,8 ; 0 ; 0 ; 0,2], B = [0,5 ; 0,1 ; 0,1 ; 0,3]
Brut : d(q,A) ≈ 0,245 < d(q,B) ≈ 0,424 → **A** (même couleur dominante)
Après √ : d(q,A) ≈ 0,632 > d(q,B) ≈ 0,579 → **B** (mêmes couleurs secondaires)

Pondération IDF

$$\mathbf{df(k) = (1/N)\ \sum_i x_i(k)} \quad \cdot \quad \mathbf{idf(k) = \ln(1\ /\ df(k))}$$

`df = X.mean(axis=0)` ; `idf = np.log(1.0 / (df + 1e-6))` (le 1e-6 évite la division par 0). `np.log` = **ln**.

TUTO : DF, IDF ET PONDÉRATION

```
X = [[0,5 ; 0,5 ; 0  ]
     [0,6 ; 0,2 ; 0,2]
     [0,4 ; 0,3 ; 0,3]]
df  = [0,5  ; 0,333 ; 0,167]
idf = [0,69 ; 1,10  ; 1,79 ]
    = [ln 2 ; ln 3  ; ln 6 ]
```

- $\sum_k$ df(k) = 1 (moyenne d'histogrammes de somme 1).
- Couleur partout (df → 1) : idf → 0. Couleur rare : idf élevé.
- TD : idf min **3,15** → df = $e^{-3,15}$ ≈ **4,3 %** ; idf max **8,36** → df ≈ 0,02 %.

`X_sqrt_idf = X_sqrt * idf[None, :]` : (1491, 128) × (1, 128), le **broadcasting** multiplie la colonne k par idf[k]. Équivalent : `X_sqrt * idf`. Erreur de forme : `idf[:, None]`, `np.diag(idf)`.

Représentation (LAB, P = 128)	mAP	Gain
Histogramme brut	40,43 %	—
√x	54,91 %	**+14,5 pts**
√x × idf	55,29 %	+0,4 pt

PIÈGES

df d'une couleur ≠ df d'un mot : ici une proportion moyenne, pas un nombre de documents. Le plus gros gain vient de **√x**, pas de l'IDF. Diviser l'histogramme par **P** au lieu du nombre de pixels est faux. La quantification se fait **en LAB** ; seul l'affichage repasse en RGB.

ASTUCE

Même démarche que pour le texte : **extraire des features** (histogramme), choisir une **métrique** (euclidienne, dans un bon espace), **trier** par similarité, puis **mesurer** (mAP) l'effet de chaque amélioration. La représentation compte autant que l'algorithme de recherche.